



Mantenimiento del CRUD pets-crud

Sistema de Gestión de Mascotas conectado a epn-event-manager

Correctivo

Preventivo

Perfectivo

Adaptativo

Objetivo de la presentación

- Presentar el diagnóstico del CRUD.
- Explicar los mantenimientos aplicados.
- Definir pruebas y evidencias.
- Justificar la mejora del sistema.

Contexto del sistema

Módulo CRUD para la gestión de mascotas dentro del repositorio YO-SOY-CRUD.

¿Qué hace el CRUD?

Permite registrar, listar, consultar, editar y eliminar mascotas. También muestra estadísticas básicas del estado de las mascotas.

Integración

El sistema se conecta con epn-event-manager para enviar eventos cuando se crea, consulta, actualiza o elimina información.

Tecnologías

Node.js, Express, SQLite, HTML, CSS, JavaScript, Axios, ESLint y node:test.

Puerto

Backend por defecto:
`http://localhost:4002`

Diagnóstico inicial

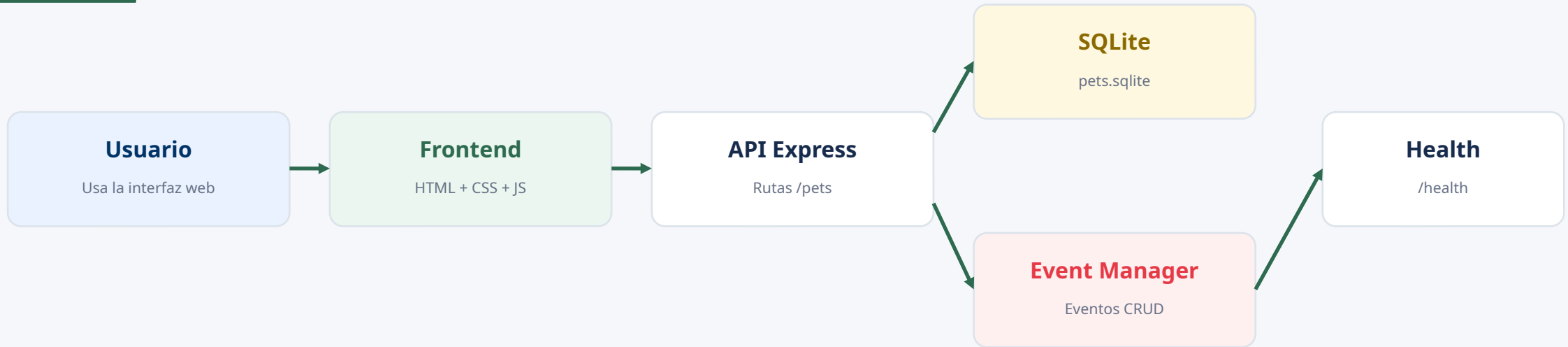
Se revisaron funcionalidad, configuración, seguridad, documentación y pruebas.



Aspecto	Estado encontrado	Riesgo
Funcionalidad CRUD	Operaciones principales implementadas	Bajo
README	Puerto documentado incorrectamente	Medio
Frontend	API fija con localhost:4002	Alto
Pruebas	Script configurado, pero sin pruebas visibles	Alto
Seguridad UI	Uso de innerHTML con datos del usuario	Alto
Event Manager	Las rutas esperan el envío del evento	Medio

Arquitectura funcional

Flujo general entre usuario, frontend, backend, base de datos y Event Manager.



La mejora busca que el flujo sea más estable, seguro y fácil de ejecutar en distintos entornos.

Problemas identificados

Los problemas se clasificaron según impacto técnico y académico.

1. Puerto inconsistente

README indicaba 4001, pero el servidor usa 4002. Esto genera confusión al ejecutar.

2. API fija

El frontend apuntaba directamente a localhost:4002. Esto dificulta usar otro puerto o equipo.

3. Falta de pruebas

Existía npm test, pero era necesario agregar casos de prueba reales.

4. Riesgo XSS

El listado usaba innerHTML con datos ingresados por el usuario.

5. Bloqueo por integración

Las operaciones esperaban el envío del evento antes de responder al usuario.

6. Documentación incompleta

Faltaban variables de entorno, evidencia de pruebas y explicación del mantenimiento.

Mantenimiento correctivo

Correcciones aplicadas para resolver errores o comportamientos problemáticos.

Antes

- README con puerto incorrecto
- API fija en el navegador
- Datos pintados con innerHTML
- Event Manager podía retrasar la respuesta
- Pruebas configuradas pero sin casos



Después

- README corregido a puerto 4002
- API adaptable con `window.location.origin`
- Escape de datos para prevenir XSS
- Envío de eventos sin bloquear la operación
- Casos de prueba agregados

Mantenimiento preventivo

Acciones para reducir riesgos futuros y facilitar nuevas revisiones.

Pruebas automatizadas

Se agregan pruebas para validar funciones críticas: validación, normalización, creación, búsqueda, actualización, eliminación y estadísticas.

Variables de entorno

Se documenta la configuración mediante `.env.example`: puerto, ruta de base de datos, URL del Event Manager y orígenes permitidos.

Calidad de código

Se mantienen comandos para revisión: `npm run check`, `npm run lint` y `npm test`. Esto permite verificar antes de entregar.

Comandos de validación

```
npm install  
npm run check  npm run lint  npm test
```

Mantenimiento perfecto

Mejoras que aumentan la calidad, claridad y experiencia de uso.



Mejora	Descripción	Beneficio
README ampliado	Incluye instalación, endpoints, pruebas y mantenimiento aplicado.	Facilita la entrega y defensa.
Mensajes de validación	Se aclara qué campos son obligatorios o inválidos.	Mejora la experiencia del usuario.
Uso de estadísticas	Se aprovecha el endpoint /pets/stats.	Evita cálculos duplicados en frontend.
Separación futura	CSS y JS podrían separarse del index.html.	Mejora mantenibilidad.

Este tipo de mantenimiento no solo corrige: también mejora la presentación, claridad y calidad del producto.

Mantenimiento adaptativo

Cambios para que el CRUD funcione en distintos entornos.

Antes

```
constAPI= http://localhost:4002';
```

Dependía de una URL fija. Si se cambiaba de puerto, servidor o equipo, podía fallar.



Después

```
constAPI= window.location.origin;
```

Se adapta al origen desde donde se abre la aplicación.

También se documenta

```
PORT  
PETS_DB_PATH  
EVENT_MANAGER_URL  
EVENT_MANAGER_HEALTH_URL  
ALLOWED_ORIGINS
```

Plan de pruebas

Pruebas necesarias para demostrar que el mantenimiento fue correcto.

Pruebas unitarias

- validatePet()
- normalizeSpecies()
- normalizeStatus()
- insertPet()
- updatePet()
- deletePetById()
- getPetStats()

Pruebas de integración

- GET /health
- GET /pets
- GET /pets/:id
- POST /pets
- PUT /pets/:id
- DELETE /pets/:id
- GET /pets/stats

Pruebas manuales

- Registrar mascota
- Editar mascota
- Eliminar mascota
- Filtrar por estado
- Buscar por texto
- Ver eventos en Event Manager
- Revisar consola sin errores

Casos de prueba principales

Ejemplos en formato Dado / Cuando / Entonces.

Caso	Dado	Cuando	Entonces
CP-01 Crear mascota válida	El usuario está en el formulario.	Ingresa nombre, especie y dueño.	El sistema guarda la mascota y la lista.
CP-02 Validar campos obligatorios	El formulario está vacío.	Presiona guardar.	El sistema muestra error y no guarda.
CP-03 Actualizar mascota	Existe una mascota registrada.	Modifica datos y guarda.	El sistema actualiza la información.
CP-04 Eliminar mascota	Existe una mascota en el listado.	Confirma la eliminación.	La mascota desaparece del listado.
CP-05 Health check	El backend está iniciado.	Accede a /health.	Devuelve status ok y estado del hub.

Evidencias recomendadas

Capturas que se pueden anexar al informe o mostrar durante la defensa.

N.º	Lugar	Evidencia
01	Terminal	npm run check sin errores
02	Terminal	npm run lint sin errores
03	Terminal	npm test con pruebas aprobadas
04	Navegador	CRUD abierto en http://localhost:4002
05	Interfaz	Crear, editar y eliminar mascota
06	API	/health, /pets y /pets/stats
07	Event Manager	Evento CREATE, UPDATE o DELETE recibido

Resultado del mantenimiento

Resumen de cambios y beneficios obtenidos.

Correctivo

Se corrigieron inconsistencias, riesgos de seguridad y comportamiento bloqueante.

Preventivo

Se agregaron pruebas, configuración documentada y comandos de validación.

Perfectivo

Se mejoró la documentación, claridad del sistema y facilidad de revisión.

Adaptativo

El CRUD queda preparado para distintos puertos, equipos y URLs de integración.

Conclusión: el CRUD era funcional, pero ahora queda mejor preparado para ser ejecutado, probado, documentado y mantenido.

Guion corto para exponer

Puedes usarlo como cierre o apoyo oral.

“En este trabajo se realizó el análisis y mantenimiento del módulo pets-crud. Se identificó que el sistema ya contaba con operaciones CRUD, base de datos SQLite e integración con epn-event-manager, pero presentaba detalles que podían afectar su uso y mantenimiento. Por eso se aplicó mantenimiento correctivo para resolver inconsistencias, preventivo mediante pruebas y configuración, perfectivo para mejorar documentación y experiencia, y adaptativo para permitir que el sistema funcione en otros entornos. Finalmente, se definieron pruebas y evidencias para comprobar que los cambios son válidos.”

Fin